Sapienza Università di Roma

Intelligenza Artificiale

libro del corso:
Artificial Intelligence: A Modern Approach
(Norvig, Russell)

aglaia norza

16 marzo 2026

thisisaglaia@gmail.com
github.com/AglaiaNorza

Indice

A.1 Agenti Intelligenti 2
A.1.1 Agenti, ambienti, razionalità 2
A.1.1.1 Tipologie di agenti 4

A.2 Esplorazione sistematica di Spazi degli Stati 8
A.2.1 Agenti risolutori di problemi 8
A.2.1.1 Formulazione del problema 8
A.2.1.2 Stato vs. Nodo 8
A.2.2 Algoritmi di Ricerca Non Informata 9
A.2.2.1 Ricerca in ampiezza (BFS - Breadth-First Search) 9
A.2.2.2 Ricerca Uniform-Cost (Min-Cost) 9
A.2.2.3 Ricerca in profondità (DFS - Depth-First Search) 9
A.2.2.4 Ricerca ad approfondimento iterativo (IDS) 9
A.2.3 Algoritmi di Ricerca Informata 10
A.2.3.1 Ricerca Best-First Greedy 10
A.2.3.2 Ricerca A^* 10

A.3 Algoritmi a Miglioramento Iterativo 12
A.3.1 Ricerca Locale 12
A.3.1.1 Hill Climbing (Ricerca in salita) 12

A.1

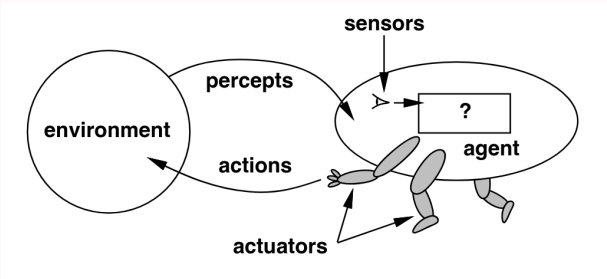
Agenti Intelligenti

A.1.1 Agenti, ambienti, razionalità

Def. 1: **Agente**

Un agente è qualsiasi sistema che:

- percepisce il suo ambiente mediante **sensori**
- agisce sull'ambiente mediante **attuatori**



Ci è utile introdurre alcuni termini specifici relativi agli agenti e ai loro ambienti:

- **percezioni** (insieme $\mathcal{P}$): input percettivi dell'agente in un dato istante
- **sequenza percettiva**: la storia di quello che l'agente ha percepito nella sua intera esistenza
- **funzione agente**: descrizione matematica del comportamento dell'agente, che può dipendere dalla sua intera sequenza percettiva (ma non da qualcosa che non ha percepito)

$$f : \mathcal{P}^* \to \mathcal{A}$$

  in cui $\mathcal{A}$ è l'insieme delle azioni che l'agente esegue
- **programma agente**: implementazione (interna al sistema fisico) della funzione agente

Def. 2: **Agente Razionale**

Definiamo (almeno per adesso) come **agente razionale** un agente che sceglie le azioni che *massimizzano il valore atteso della sua misura di prestazione*, data la sequenza di percezioni e ogni altra sua conoscenza pregressa.

La **misura di prestazione** è una misurazione oggettiva della desiderabilità della sequenza di

stati attraversati dall'ambiente a seguito delle azioni dell'agente.

- deve essere valutata oggettivamente sugli stati dell'ambiente, non dell'agente
- di solito è preferibile una misura che considera gli effetti delle azioni piuttosto che le azioni stesse

Esempio: agente aspirapolvere

Consideriamo un semplice agente aspirapolvere che si muove in un ambiente composto da due mattonelle. Un agente reattivo che aspira se la mattonella dove si trova è sporca e si sposta nell'altra mattonella se è pulita risulta razionale rispetto a una misura di prestazione che assegna semplicemente un punteggio per ogni riquadro pulito.

```
function agente_reattivo_aspirapolvere([posizione, stato]):
    if stato = Sporco then return Aspira;
    else
        if posizione = A then return Destra;
        else return Sinistra; /* posizione = B */
```

Tuttavia, un agente simile continuerà a muoversi in eterno tra una mattonella e l'altra anche quando entrambe sono pulite. Se la misura di prestazione fosse formulata in modo diverso, introducendo ad esempio penalità per ogni movimento effettuato, per il consumo di elettricità o per il rumore prodotto, questo stesso agente non sarebbe più razionale a causa del suo inutile dispendio di energie

Il primo passo nella progettazione di un agente è la definizione del suo **task environment**. Un task environment è composto da:

- **P**erformance (misura di prestazione)
- **E**nvironment (ambiente)
- **A**ctuators (attuatori)
- **S**ensors (sensori)

Esempi di PEAS

Agente	**P**restazione	**E**nvironment	**A**ttuatori	**S**ensori
Taxi automatico	Sicurezza, velocità, legalità, comfort, profitto	Strade, altri veicoli, pedoni, clienti	Sterzo, acceleratore, freni, frecce, sintesi vocale	Telecamere, sonar, tachimetro, GPS, accelerometro, microfono
Diagnosi medica	Guarigione, costi minimi, assenza di denunce	Paziente, staff medico	Schermo	Tastiera
Shopping online	Prezzo, qualità, attinenza ai desideri	Siti web dei negozi, corrieri	Schermo, vai-a-URL, riempimento form.	Pagine HTML da interpretare

È possibile classificare gli ambienti secondo alcune coordinate importanti per la progettazione di agenti:

Dimensione	Tipi di ambienti	Cosa riguardano
Osservabilità	Totalmente, parzialmente, inosservabile	Capacità dei sensori di dare accesso a tutte le proprietà rilevanti dello stato dell'ambiente in ogni momento.
Agenti	Mono-agente, multi-agente	Presenza di altri agenti che possono *cooperare* o *competere* per obiettivi o risorse.
Determinismo	Deterministico, stocastico, non-deterministico	Indica se lo stato successivo è determinato univocamente da stato corrente e azione (*stocastico*: incertezza gestita con distribuzioni di probabilità; *non-deterministico*: azioni definite dal set di risultati possibili, spesso richiede successo per *tutti* gli esiti)
Episodicità	Episodico, sequenziale	Specifica se l'esperienza è divisibile in episodi atomici indipendenti o se le scelte influenzano il futuro.
Dinamicità	Statico, dinamico, semi-dinamico	Indica se l'ambiente (*dinamico*) o la misura di prestazione (*semi-dinamico*) muta mentre l'agente riflette.
Continuità	Discreto, continuo	Indica se stato, ambiente, percezioni, azioni sono discrete (interi) o continue (reali). Un ambiente continuo può essere discretizzato
Conoscenza	Noto, ignoto	Denota la conoscenza a priori delle regole "fisiche" dell'ambiente e delle conseguenze delle azioni.

A.1.1.1 Tipologie di agenti

Ci sono quattro principali **tipi di agente**:

- agenti reattivi semplici
- agenti reattivi basati su modello
- agenti basati su obiettivi
- agenti basati sull'utilità

Ad ognuno di questi tipi è possibile aggiungere la capacità di *apprendere*.

A.1.1.1.1 Agenti reattivi semplici

Le azioni degli agenti reattivi semplici sono determinate esclusivamente dalla **percezione corrente**.

```
function a_r_aspirapolvere([posizione, stato]): azione
    if stato = Sporco then return Aspira;
    else
        if posizione = A then return Destra;
        else return Sinistra;
```

Gli agenti reattivi semplici seguono una "condition-action rule" (`if condizione then azione`)

- è utile che agenti complessi abbiano parti puramente reattive

A.1.1.1.2 Agenti reattivi basati su modello

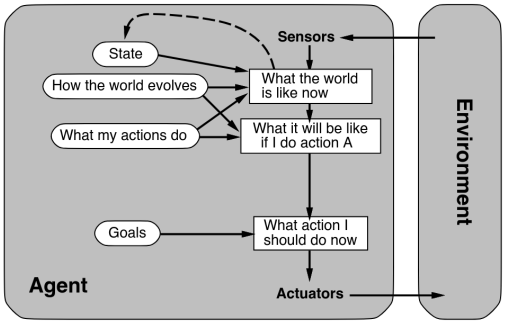
Le azioni degli agenti reattivi basati su modello sono detereminate dalla percezione corrente e da un "modello del mondo", una rappresentazione interna della parte del mondo che l'agente non può vedere all'istante corrente.

Il modello deve essere conosciuto a priori (in qualche modo "hard-coded" nell'agente)

```
function MODEL-BASED-REFLEX-AGENT(percept): action
  persistent: state, the agent's current conception of the world state
              model, a description of how the next state depends on current state and
                  action
              rules, a set of condition-action rules
              action, the most recent action, initially none

  state ← UPDATE-STATE(state, action, percept, model)
  rule ← RULE-MATCH(state, rules)
  action ← rule.ACTION
  return action
```

A.1.1.1.3 Agenti basati su obiettivi

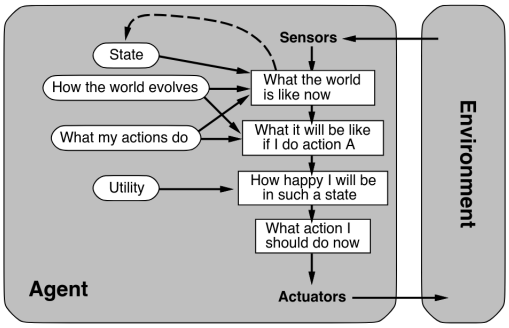


In questa tipologia, l'azione è determinata dalla percezione corrente, dallo stato e dagli obiettivi dell'agente (la sola conoscenza dello stato e delle percezioni spesso non è sufficiente per prendere una decisione)

> Consideriamo un pilota di taxi - se la percezione corrente è un incrocio, l'agente non può decidere se girare a destra o a sinistra senza conoscere la destinazione finale; l'obiettivo fornisce questa informazione.

Non sempre l'obiettivo è raggiungibile tramite una singola azione. In questi casi l'agente deve ricorrere a tecniche di **ricerca e pianificazione** per calcolare intere sequenze di azioni necessarie per raggiungere l'obiettivo desiderato (l'agente sa come sarà il mondo dopo ogni possibile mossa, e sceglie quelle che lo portano all'obiettivo).

A.1.1.1.4 Agenti basati sull'utilità



L'azione, in questo caso, è determinata dalla percezione corrente, dallo stato e dagli obiettivi dell'agente, ed è esplicitamente mirata a raggiungere un'alta **utilità**.

Mentre l'obiettivo definisce uno stato da raggiungere, l'utilità fornisce un criterio per valutare *come* arrivarci.

> Se ci sono più percorsi per arrivare alla meta, l'agente pilota di taxi dovrà chiedersi quale sia il percorso più veloce/affidabile/sicuro/ecc.

La funzione di utilità rappresenta un'interiorizzazione della misura di prestazione dell'ambiente (considera quanto sono "buoni" gli stati intermedi), e aiuta concretamente l'agente a comportarsi in modo razionale.

Se l'agente si trova a operare in ambienti non totalmente osservabili o non deterministici, esso agirà in modo da massimizzare l'**utilità attesa** (ovvero la media ponderata delle utilità dei vari risultati possibili).

A.1.1.1.5 Agenti in grado di apprendere

Gli agenti visti in precedenza funzionano bene solo se gli scenari rimangono sempre gli stessi. Se si introducono per esempio elementi *greedy*, il programma può sbagliare; per questo motivo è necessario introdurre un elemento di apprendimento che permetta all'agente di esplorare e sperimentare su stati **nuovi**.

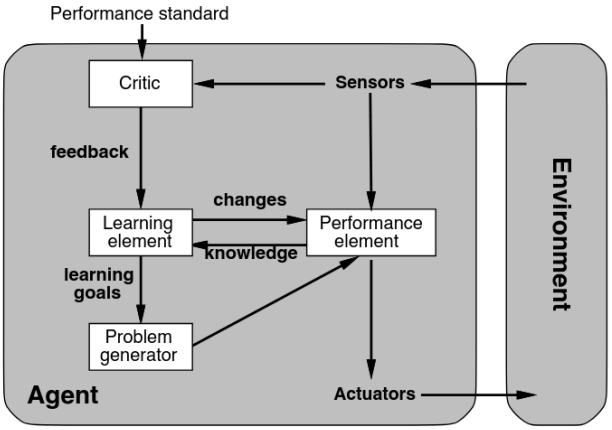
Ad ognuno dei tipi di agenti finora descritti è possibile aggiungere la capacità di apprendere. In generale, l'apprendimento negli agenti intelligenti può essere riassunto come un processo di modifica di ogni componente dell'agente per allinearli maggiormente alle informazioni di feedback disponibili, migliorandone così le prestazioni complessive.

L'architettura di un agente con capacità di apprendimento è suddivisa in quattro componenti fondamentali:

- **Performance element** (elemento esecutivo): l'"agente" vero e proprio descritto in precedenza (quello che riceve le percezioni e seleziona le azioni) - a questo punto, però, non sarà più costituito da codice statico, ma da una **struttura dati di rappresentazione**, in modo che il suo comportamento possa essere effettivamente modificato e aggiornato.
- **Critic** (elemento critico): Valuta le azioni compiute dall'agente confrontandole con un *performance standard* (standard di prestazione) e fornisce un *feedback*. È necessario perché le percezioni di per sé non indicano all'agente se ha avuto successo (ad esempio, un programma di scacchi sa di aver fatto scacco matto dalle percezioni, ma ha bisogno di uno standard per sapere che "vincere" è un evento positivo).

  - Lo standard di prestazione deve essere **fisso** e considerato concettualmente esterno all'agente: l'agente non deve poterlo modificare per adattarlo al proprio comportamento.
  - Distingue parte della percezione in entrata sotto forma di **premio** (*reward*) o **penalità** (*penalty*), fornendo un feedback diretto sulla qualità del comportamento.

- **Learning element** (elemento di apprendimento): riceve il feedback dal *critic* e conosce il programma agente. Il suo compito è suggerire i cambiamenti per generare modifiche nel modo di comportarsi dell'agente. Inoltre, acquisisce e memorizza nuova conoscenza (*knowledge*) sull'ambiente.
  - L'osservazione degli stati successivi all'azione consente all'agente di imparare "cosa fanno le azioni" e "come evolve il mondo" (ad esempio, capire quanto il taxi riesca effettivamente a frenare su una strada bagnata).
- **Problem generator** (generatore di problemi): genera nuovi obiettivi di apprendimento (*learning goals*) suggerendo azioni che portino ad esperienze nuove e "formative". È il componente che spinge l'agente a uscire dalla sua routine per sperimentare su stati nuovi.
  - Se l'elemento esecutivo fosse lasciato solo, continuerebbe a eseguire all'infinito unicamente le azioni che ritiene migliori in base alle sue limitate conoscenze attuali.
  - Il problem generator spinge l'agente a esplorare, compiendo azioni che nel breve termine potrebbero sembrare *sub-ottimali*, ma che portano a scoprire alternative migliori per il lungo termine.



A.2

Esplorazione sistematica di Spazi degli Stati

A.2.1 Agenti risolutori di problemi

Gli agenti risolutori di problemi sono agenti basati su obiettivi che considerano gli stati come **entità atomiche** (prive di struttura interna). Operano tipicamente in ambienti discreti, totalmente osservabili, noti, deterministici e statici.

In queste condizioni favorevoli, l'agente può calcolare in anticipo un'intera sequenza di azioni (**piano**) per raggiungere l'obiettivo ed eseguirla "ad occhi chiusi", certo del risultato.

A.2.1.1 Formulazione del problema

Def. 3: **Problema di ricerca**

Un problema di ricerca è definito da cinque componenti:

- **Stato iniziale**: lo stato in cui l'agente inizia la ricerca.
- **Azioni**: l'insieme delle azioni possibili in un dato stato s, denotato come $Azioni(s)$.
- **Modello di transizione**: funzione $Risultato(s, a)$ che descrive la fisica dell'ambiente, restituendo lo stato risultante dall'esecuzione dell'azione a nello stato s.
- **Test obiettivo**: condizione o funzione che determina se un dato stato soddisfa l'obiettivo.
- **Costo di cammino**: funzione che assegna un costo numerico a ogni cammino (solitamente calcolata come la somma dei costi di passo).

Una **soluzione** è una sequenza di azioni che individua un cammino dallo stato iniziale a uno stato obiettivo. Una **soluzione ottima** è la soluzione a costo minimale.

A.2.1.2 Stato vs. Nodo

Durante l'esplorazione, l'algoritmo mantiene una **frontiera** di ipotesi aperte. È fondamentale distinguere tra stato e nodo:

- **Stato**: una rappresentazione di una configurazione del mondo (spesso un'"etichetta")
- **Nodo**: struttura dati componente dell'albero di ricerca. Oltre allo stato, un nodo include il puntatore al nodo padre, l'azione generatrice, la profondità e il costo di cammino $g(n)$ calcolato fino a quel punto.

> Ricerca su albero vs su grafo
>
> La presenza di cammini ciclici o stati ripetuti può generare un albero di ricerca infinito anche in problemi con uno spazio degli stati finito. Per ovviare a ciò, la ricerca su grafo memorizza gli stati già visitati in un insieme **esplorati** (closed set), espandendo solo nodi che portano a stati non ancora visitati o non presenti in frontiera.

A.2.2 Algoritmi di Ricerca Non Informata

Le strategie non informate possiedono solo la formulazione del problema; l'algoritmo può solo generare successori e testare l'obiettivo, senza sapere se uno stato sia "meglio" di un altro in prospettiva.

A.2.2.1 Ricerca in ampiezza (BFS - Breadth-First Search)

- **Strategia**: espande il nodo più vicino alla radice. Usa una coda FIFO per la frontiera.
- **Completezza**: sì, se il fattore di ramificazione b è finito.
- **Ottimalità**: se il costo di passo è costante (sse un cammino è funzione monotona della profondità)
- **Complessità**: tempo e spazio $O(b^d)$ (dove d è la profondità della soluzione)

  il problema più grande è l'occupazione di memoria, che cresce esponenzialmente

A.2.2.2 Ricerca Uniform-Cost (Min-Cost)

- **Strategia**: espande il nodo con il costo di cammino $g(n)$ minimo. La frontiera è una coda di priorità.
- **Completezza/Ottimalità**: se b è finito e tutti i costi di passo sono $\geq \epsilon > 0$.
- **Complessità**: tempo e spazio $O(b^{1+\lfloor C^*/\epsilon \rfloor})$

A.2.2.3 Ricerca in profondità (DFS - Depth-First Search)

- **Strategia**: espande sempre il nodo più profondo. Usa una coda LIFO (pila).
- **Completezza**: solo in spazi finiti e se evita i cammini ridondanti (ricerca su grafo). Si perde facilmente in rami infiniti.
- **Ottimalità**: no.
- **Complessità**: tempo $O(b^m)$ (dove m è la profondità massima dell'albero)

  il vantaggio principale è lo Spazio: se implementata ad albero, richiede solo $O(bm)$

A.2.2.4 Ricerca ad approfondimento iterativo (IDS)

L'IDS esegue iterativamente ricerche a profondità limitata, aumentando il limite ad ogni passaggio $(0, 1, 2, \dots)$ finché non trova l'obiettivo.

- **Pro**: unisce i vantaggi di BFS e DFS. È completa e ottima (come BFS) ma richiede una memoria lineare $O(bd)$ (come DFS).

- **Tempo**: $O(b^d)$ - la ripetizione dei livelli superiori sembra uno spreco, ma incide poco asintoticamente in quanto la maggior parte dei nodi si trova nei livelli più bassi (vengono ripetuti "pochi" nodi, solo quelli più vicini alla radice)

A.2.3 Algoritmi di Ricerca Informata

Gli algoritmi di ricerca informata possiedono conoscenza specifica del dominio, incarnata in una **funzione euristica** $h(n)$.

- $h(n)$: stima del costo minimo per raggiungere l'obiettivo dal nodo n. È 0 se il nodo è una soluzione, e > 0 altrimenti.

A.2.3.1 Ricerca Best-First Greedy

- **Strategia**: estrae dalla frontiera il nodo che *sembra* più vicino all'obiettivo, ordinando i nodi esclusivamente per $h(n)$.
- **Limitazioni**: dimentica il passato (il costo già sostenuto) e non è garantita per trovare soluzioni ottime; si tratta l'approccio greedy può rivelarsi fuorviante.

A.2.3.2 Ricerca A^*

L'algoritmo A^* risolve i problemi del Best-First Greedy combinando il costo effettivo passato e la stima futura. La frontiera è ordinata in base alla funzione di valutazione:

$$f(n) = g(n) + h(n)$$

Dove $f(n)$ rappresenta il costo stimato del cammino più economico che passa per n.

A.2.3.2.1 Ottimalità di A^*

La garanzia che A^* restituisca una soluzione ottima dipende strettamente dalle proprietà dell'euristica $h(n)$.

- **Ammissibilità**: un'euristica è *ammissibile* se non sovrastima mai il costo per raggiungere l'obiettivo ($h(n) \leq$ costo reale minimo). È in pratica una stima ottimistica.

> Thm. 1: **Euristica ammissibile su albero**
>
> Con h ammissibile, A^* su *albero* trova sempre la soluzione ottima

- **Consistenza** (o monotonia): un'euristica è *consistente* se rispetta la disuguaglianza triangolare: per ogni nodo n e suo successore n', vale: $h(n) \leq costo(n, n') + h(n')$.

  $\rightarrow$ un'euristica consistente è implicitamente anche ammissibile

> Thm. 2: **Euristica consistente**
>
> Con h consistente, A^* su *grafo* trova sempre la soluzione ottima. I valori di $f(n)$ lungo un cammino risultano non decrescenti.

Il problema principale di A^* è la complessità spaziale.

A.2.3.2.2 Sviluppare Euristiche

Se si dispongono di due euristiche ammissibili h_1 e h_2, e per ogni nodo $h_2(n) \geq h_1(n)$, si dice che h_2 **domina** h_1. L'euristica dominante (a valori più alti) è preferibile poiché farà espandere ad A^* un numero minore o uguale di nodi.

Le euristiche ammissibili spesso si derivano calcolando il costo esatto di una soluzione in una versione rilassata (con meno vincoli) del problema originale.

A.3

Algoritmi a Miglioramento Iterativo

A.3.1 Ricerca Locale

Negli algoritmi di ricerca visti finora, l'obiettivo era trovare un *cammino* verso la soluzione. In molti problemi di ottimizzazione, tuttavia, il percorso per raggiungere la meta è irrilevante: ciò che conta è solo lo **stato finale** (es. problema n-queens).

In questi casi si usano algoritmi di **ricerca locale** che operano utilizzando una formulazione a *stato completo*: partono da una configurazione sub-ottimale e applicano modifiche iterative per migliorarne la qualità (o minimizzarne il costo), esplorando solo il vicinato dello stato corrente.

Vantaggi principali:

- usano pochissima memoria (spesso tengono traccia di un solo nodo corrente)
- possono trovare soluzioni ragionevoli in spazi degli stati infiniti o continui dove gli algoritmi sistematici classici fallirebbero

A.3.1.1 Hill Climbing (Ricerca in salita)

È un algoritmo di tipo **greedy** che si sposta sempre in direzione di un valore crescente (o decrescente, se sta minimizzando un costo), muovendosi verso il "vicino" migliore. Termina quando raggiunge un "picco" ("massimo locale") in cui nessun vicino ha un valore più alto.

> "cercare di raggiungere la cima del monte Everest in una fitta nebbia, soffrendo di amnesia".

- **Problemi ricorrenti**:
  - **massimi locali**: picchi che sono più alti dei vicini immediati, ma più bassi del massimo globale - l'algoritmo si blocca
  - **creste (ridges)**: sequenze di massimi locali non connesse direttamente tra loro - l'algoritmo è costretto a "oscillare" e procede a fatica
  - **plateaux**: aree piatte dello spazio degli stati; possono essere massimi locali piatti o "spalle" da cui è ancora possibile salire - l'algoritmo finisce per vagare casualmente

```
function hill_climbing(problema): stato ottimo locale
  nodo_corrente <- crea_nodo(problema.stato_iniziale);
  while true do
    mossa_trovata <- false;
    while mossa_trovata = false and nodo_corrente ha ancora vicini da considerare do
      vicino <- prossimo 'vicino' di nodo_corrente;
```

```
    if vicino è migliore o della stessa qualità di nodo_corrente then
      /* effettua mosse laterali */
      nodo_corrente <- vicino;
      mossa_trovata <- true;
  if mossa_trovata = false then
    return nodo_corrente.stato;
```

Listing A.3.1: Hill-climbing con prima scelta